

Documentación del Sistema de Trading Automatizado para Binance

Cero

21 de octubre de 2025

Índice

1. Introducción	2
1.1. Objetivos del Proyecto	2
2. Arquitectura del Proyecto	2
3. Configuración Inicial	3
3.1. Prerrequisitos	3
3.2. Instalación de Dependencias	3
3.3. Archivo de Configuración (.env)	3
4. Guía de Uso	4
4.1. Ejecución en Modo Real (Live Trading)	4
4.2. Backtesting y Simulación de Estrategias	4
4.2.1. Paso 1: Descargar Datos Históricos	4
4.2.2. Paso 2: Detección de Señales	4
4.2.3. Paso 3: Simulación de Resultados	5
4.3. Visualización con el Dashboard	5
5. Despliegue en Servidor	5
5.1. Uso de screen	5

1. Introducción

Este documento describe la arquitectura, configuración y uso del sistema de trading automatizado diseñado para interactuar con la API de Binance. El sistema está compuesto por un conjunto de scripts de Python que permiten el análisis de mercado en tiempo real, la ejecución de estrategias de trading, el backtesting y la visualización de datos.

1.1. Objetivos del Proyecto

- **Monitorización:** Analizar datos de mercado (precios, volumen) y calcular indicadores técnicos (EMAs, RSI, Bandas de Bollinger, etc.).
- **Detección de Señales:** Identificar patrones de velas y configuraciones estratégicas (Cruce Dorado, Wyckoff, etc.) para generar señales de compra (LONG) y venta (SHORT).
- **Gestión de Riesgo:** Calcular automáticamente los niveles de Stop Loss y Take Profit para cada operación.
- **Notificaciones:** Enviar alertas en tiempo real a través de Telegram.
- **Visualización:** Ofrecer un dashboard interactivo para visualizar gráficos, indicadores y operaciones ejecutadas.
- **Backtesting:** Probar la efectividad de las estrategias utilizando datos históricos.

2. Arquitectura del Proyecto

El sistema se compone de varios scripts de Python, cada uno con una responsabilidad específica:

`common_utils.py` El corazón del sistema. Contiene la lógica compartida, incluyendo la obtención de datos, cálculo de indicadores, gestión de riesgo centralizada, envío de notificaciones y las funciones que manejan el bucle de ejecución principal de los bots.

`trading-v6.py` Bot de trading que implementa estrategias basadas en cruces de medias móviles (Golden/Death Cross) y patrones de velas. Actúa como un "módulo de estrategia" que se conecta al motor principal de `common_utils.py`.

`trading-wyckoff.py` Bot alternativo que implementa una estrategia simplificada basada en los conceptos de Wyckoff (Spring/Upthrust) y patrones de velas. También se conecta al motor principal.

`simulate_trades.py` Herramienta de simulación que lee las señales generadas en un backtest (desde un archivo `*_backtest.csv`) y, utilizando datos históricos, calcula el resultado de cada operación (victoria/pérdida), el P&L, y la evolución de un balance de cuenta.

`dashboard.py` Aplicación web interactiva (Streamlit) que sirve como centro de control visual. Permite monitorizar el mercado en vivo y realizar un análisis exhaustivo de los resultados del backtesting, incluyendo métricas avanzadas, curvas de capital y gestión de presets.

`download_data.py` Herramienta para descargar grandes volúmenes de datos históricos de Binance y guardarlos en un archivo CSV, esencial para el proceso de backtesting.

`.env` Archivo de configuración para almacenar variables de entorno como claves de API, parámetros de estrategia y tokens de Telegram.

`trades_log.csv` Archivo donde se registran las **oportunidades de trading** generadas por los bots en **modo real (live)**.

`*_backtest.csv` Archivos generados por los bots al ejecutarse en modo backtest. Contienen las señales detectadas en los datos históricos.

`*_results.csv` Archivos generados por `simulate_trades.py` que contienen el resultado final y el análisis de rendimiento de cada operación simulada.

3. Configuración Inicial

3.1. Prerrequisitos

Asegúrate de tener instalado Python 3.8 o superior y pip.

3.2. Instalación de Dependencias

Instala todas las librerías necesarias ejecutando:

```
1 pip install -r requirements.txt
```

3.3. Archivo de Configuración (.env)

Crea un archivo llamado `.env` en la raíz del proyecto con el siguiente contenido y ajústalo a tus necesidades:

```
1 # --- Parámetros Generales ---
2 SYMBOL="BTCUSDT"
3 INTERVAL="1h"
4 LIMIT=250
5 LOG_LEVEL="INFO"
6 SLEEP=3600 # Segundos de espera entre ciclos (1 hora)
7
8 # --- Parámetros de Indicadores y Patrones ---
9 VOLUME_SMA_PERIOD=20
10 ATR_WINDOW=14
11 BOLLINGER_WINDOW=20
12 POC=65000.0
13 HAMMER_MULTIPLIER=2.0
14 SHOOTING_STAR_MULTIPLIER=2.0
15 VOLUME_MULTIPLIER=1.5
16
17 # --- Parámetros de Gestión de Riesgo ---
18 RR_RATIO=2.0 # Usado por trading-v6 para calcular TP (ej: TP = riesgo *
19             2.0)
20 SL_BUFFER=0.002 # Usado por trading-v6 para calcular SL (ej: SL = low *
                (1 - 0.002))
21 RISK_STOP_MULT=1.5 # Usado por trading_wyckoff para calcular SL (ej: SL
                    = entrada - ATR * 1.5)
```

```

21 RISK_TP_MULT=2.5 # Usado por trading_wyckoff para calcular TP (ej: TP =
    entrada + ATR * 2.5)
22
23 # --- Parámetros de Wyckoff (para trading_wyckoff.py) ---
24 WYCKOFF="true"
25 WYCKOFF_VOLUME_MULT=1.3
26
27 # --- Logs y Notificaciones ---
28 TRADES_LOG_FILE="trades_log.csv"
29 TELEGRAM_TOKEN="TU_TOKEN_DE_TELEGRAM"
30 TELEGRAM_CHAT_ID="TU_CHAT_ID"
31
32 # --- Backtesting ---
33 BACKTEST="false"
34 BACKTEST_FILE="historical_data.csv"

```

4. Guía de Uso

4.1. Ejecución en Modo Real (Live Trading)

Para ejecutar uno de los bots en modo de operación real, monitorizando el mercado y enviando señales, utiliza los siguientes comandos.

Bot principal (v6):

```
1 python trading-v6.py
```

Bot con estrategia Wyckoff:

```
1 python trading_wyckoff.py --wyckoff
```

4.2. Backtesting y Simulación de Estrategias

El proceso de backtesting se ha dividido en tres fases claras para un análisis robusto.

4.2.1. Paso 1: Descargar Datos Históricos

Usa `download_data.py` para obtener un archivo CSV con los datos de mercado necesarios. Este archivo será la base para los siguientes dos pasos.

```
1 python download_data.py --symbol ETHUSDT --interval 4h --limit 2000 --
    output eth_4h_data.csv
```

4.2.2. Paso 2: Detección de Señales

Ejecuta el bot deseado en modo backtest. El bot analizará el archivo de datos históricos y registrará todas las **oportunidades de trading** que encuentre en un nuevo archivo `*_backtest.csv`.

```

1 # Para trading-v6.py
2 # Generará 'trades_log_backtest.csv'
3 python trading-v6.py --backtest --backtest-file eth_4h_data.csv
4
5 # Para trading_wyckoff.py

```

```

6 # También generará 'trades_log_backtest.csv' (sobrescribiendo el
  anterior)
7 python trading_wyckoff.py --backtest --backtest-file eth_4h_data.csv --
  wyckoff

```

4.2.3. Paso 3: Simulación de Resultados

Una vez generadas las señales, `simulate_trades.py` las procesa. Lee el archivo `*_backtest.csv` y, usando los datos históricos, simula el resultado de cada operación para generar un informe de rendimiento completo.

```

1 python simulate_trades.py --trades-file trades_log_backtest.csv --data-
  file eth_4h_data.csv --initial-balance 10000 --risk-per-trade 0.01

```

Este comando simula una cuenta que empieza con \$10,000 y arriesga el 1 % del capital en cada operación. El resultado se guarda en un nuevo archivo, `trades_log_backtest_results.csv`.

4.3. Visualización con el Dashboard

Para iniciar el dashboard interactivo, ejecuta:

```

1 streamlit run dashboard.py

```

Accede a la URL proporcionada en la terminal (normalmente `http://localhost:8501`) para ver los gráficos. El dashboard se actualizará automáticamente y mostrará las operaciones del archivo `trades_log.csv`.

Análisis de Backtest: La pestaña **Análisis de Backtest** es una potente herramienta para evaluar estrategias. Permite cargar los archivos `*_results.csv` para visualizar métricas de rendimiento detalladas (Profit Factor, Win Rate, Max Drawdown, etc.), una curva de capital, un histograma de P&L y análisis por tipo de señal. También incluye una función para guardar y cargar "presets" de filtros.

5. Despliegue en Servidor

Para que los scripts se ejecuten de forma continua en un servidor, se recomienda usar `screen` o, para una solución más robusta, un servicio de `systemd`.

5.1. Uso de screen

1. Iniciar una nueva sesión: `screen -S trading-bot`
2. Ejecutar el script dentro de la sesión: `python trading-v6.py`
3. Desprenderse de la sesión: `Ctrl+A` y luego `d`.
4. Para volver a la sesión: `screen -r trading-bot`